

Chart Digitizer with Groq API (Free Vision Models)

Extract data points from chart images using **Groq’s free vision models**. No Claude subscription needed — Groq offers generous free tiers with fast inference.

Why Groq?

Feature	Claude Opus 4.7	Groq Vision
Cost	\$5/MTok input	Free tier available
Speed	~30-120s	~5-15s (ASIC hardware)
Vision	3.75MP, 2576px	33MP, 20MB per image
Tool Use	Native	Llama-4-Scout supports tools
Setup	Complex (beta headers, adaptive thinking)	Simple OpenAI-compatible API

Supported Models

Model Key	Full ID	Best For
llama-4-scout	meta-llama/llama-4-scout-17b-16e-instruct	Default —Tool use + JSON mode
qwen-27b	qwen/qwen3.6-27b	Multimodal with thinking mode
llama-3.2-90b	llama-3.2-90b-vision-preview	High quality (legacy)
llama-3.2-11b	llama-3.2-11b-vision-preview	Fast, lightweight (legacy)

Quick Start

1. Get Your Free Groq API Key

- 1. Go to console.groq.com/keys
- 2. Sign up (free)
- 3. Create an API key — it starts with gsk_

2. Install Dependencies

```
pip install -r requirements.txt
```

Or manually:

```
pip install groq python-dotenv pandas Pillow
```

3. Set Up Your API Key

Create a .env file in the same directory:

```
GROQ_API_KEY=gsk_your_key_here
```

4. Run the Digitizer

```
# Basic usage (uses llama-4-scout by default)
python groq_chart_digitizer.py chart.png

# Custom output filename
python groq_chart_digitizer.py chart.png my_data.csv

# Use a different model
python groq_chart_digitizer.py --model llama-3.2-90b chart.png

# Generate a test chart first
python groq_chart_digitizer.py --create-test
python groq_chart_digitizer.py test_chart.png

# Custom prompt for better accuracy
python groq_chart_digitizer.py chart.png --prompt "Y-axis is in millions. Be precise."

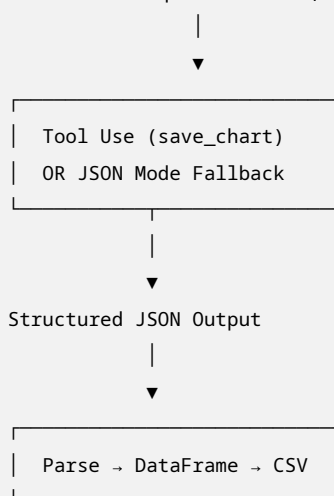
# Debug mode (verbose logging)
python groq_chart_digitizer.py --debug chart.png
```

File Structure

groq_chart_digitizer.py	# Main script (single file, ~550 lines)
test_chart.png	# Sample line chart (for testing)
test_bar_chart.png	# Sample bar chart (for testing)
requirements.txt	# Python dependencies
.env.example	# API key template
README.md	# This file

How It Works

Chart Image → Base64 Encode → Groq Vision API (Llama-4-Scout)



Key Differences from Claude Version

Aspect	Claude (Original)	Groq (This Version)
Client	AsyncAnthropic	Groq (sync, simple)
Image format	Base64 object	Data URL string
Streaming	Required for long responses	Not needed (fast enough)
Thinking	thinking={"type": "adaptive"}	Not needed
Effort	output_config={"effort": "xhigh"}	Not needed
Task budget	task_budget beta header	Not needed
Tool schema	Anthropic format	OpenAI function format
Headers	Beta headers required	None needed
Async	async/await required	Synchronous

Architecture

The script follows a 6-step pipeline:

1. **Image Encoding** — Base64 encode + build data URL + size validation (Groq limit: 4MB base64, 33MP resolution)
2. **Message Building** — OpenAI-compatible multimodal payload (text + image_url)
3. **API Call** — Groq chat.completions with tool use (function calling)
4. **Response Parsing** — Extract tool_calls or fallback to JSON from content
5. **Parse & Save** — JSON → DataFrame → CSV with rich metadata
6. **Orchestrator** — CLI with argparse, model selection, debug mode

Debugging Guide

Error: GROQ_API_KEY not found

Fix:

```
# Create .env file
echo "GROQ_API_KEY=gsk_your_key" > .env

# Or set directly (Windows)
set GROQ_API_KEY=gsk_your_key

# Or set directly (Mac/Linux)
export GROQ_API_KEY=gsk_your_key
```

Error: ModuleNotFoundError: No module named 'groq'

Fix:

```
pip install groq
```

Error: 413 Request Entity Too Large

Cause: Image exceeds Groq’s 4MB base64 limit.

Fix: Compress or resize the image:

```
# Quick resize with Pillow
```

```
from PIL import Image
img = Image.open("big_image.png")
img.thumbnail((1500, 1500)) # Resize to fit
img.save("resized.png")
```

Error: Image resolution exceeds 33 megapixels

Fix: Resize the image to be under 33MP total pixels.

Error: ValueError: Model did not use tools

Cause: The model returned text instead of calling the function.

Fix:

- 1. Use `--model llama-4-scout` (best tool support)
- 2. Add `--prompt "You MUST use the save_chart_data function"`
- 3. Try `--no-tools` flag to use JSON mode instead

Error: JSONDecodeError on tool arguments

Fix: The model may have malformed JSON. Try:

- 1. Lower temperature: edit `TEMPERATURE = 0.0` in the script
- 2. Use a different model
- 3. Simplify the chart image

Extracted values seem wrong

Symptom	Likely Cause	Fix
Values off by 10x/1000x	Missing unit conversion	Use <code>--prompt "Y-axis is in millions, convert to full numbers"</code>
Missing data points	Image too complex	Crop to just the chart area
Wrong axis labels	Low contrast / small text	Increase image resolution
Categorical X-axis wrong	Model confused	Use <code>--prompt "X-axis categories are Q1, Q2, Q3, Q4"</code>

Groq Free Tier Limits

As of 2026, Groq offers:

- **Free tier:** Generous requests per day (varies by model)
- **Rate limits:** Check your dashboard at console.groq.com

- No credit card required for free tier

Comparison: Groq vs Claude for Chart Digitization

Test	Claude Opus 4.7	Groq Llama-4-Scout
Accuracy	Excellent (1:1 pixel mapping)	Very Good
Speed	30-120s	5-15s
Cost	~\$0.50-2.00 per chart	Free
Tool forcing	Reliable	Good (Llama-4)
Complex charts	Best	Good
Setup complexity	High	Low

Recommendation: Use Groq for prototyping, testing, and simple-to-moderate charts. Use Claude for production-grade precision on complex technical diagrams.

Advanced Usage

Force JSON Mode (No Tools)

If tool use isn't working with your model:

```
python groq_chart_digitizer.py chart.png --no-tools
```

Use Different Model

```
# Fastest
python groq_chart_digitizer.py --model llama-3.2-11b chart.png

# Highest quality (legacy)
python groq_chart_digitizer.py --model llama-3.2-90b chart.png

# Multimodal with thinking
python groq_chart_digitizer.py --model qwen-27b chart.png
```

Batch Processing

```
import subprocess

charts = ["chart1.png", "chart2.png", "chart3.png"]
for chart in charts:
    subprocess.run(["python", "groq_chart_digitizer.py", chart])
```

License

MIT — Free to use and modify.